

Relatorio de Análise de Algoritmos de Ordenação de Introdução à Ciência da Computação II

Leonardo Kenzo Tanaka e Pedro Teidi de Sá Yamacita

18 de novembro de 2025

1 Introdução

O problema proposto consistiu em comparar o desempenho dos seguintes algoritmos de ordenação: Bubble Sort, Selection Sort, Insertion Sort, Shell Sort, Quick Sort (com mediana de 3 para o pivô), Heap Sort, Merge Sort, Contagem dos Menores e Radix Sort.

A solução envolveu a implementação de cada algoritmo com a instrumentação de contadores no código para obter as estatísticas desejadas:

- Tempo de Execução: O tempo que o algoritmo levou para ordenar o vetor.
- Comparações de Chaves: O número de vezes que elementos foram comparados.
- Movimentações de Registros: O número de vezes que elementos foram trocados ou movidos.

Os testes foram realizados com vetores de entrada de números inteiros nos tamanhos $N = 100, 1.000, 10.000$ e 100.000 elementos. Para cada tamanho, foram consideradas três características de vetor: ordenado, inversamente ordenado e aleatório. No caso dos vetores aleatórios, a média das estatísticas foi calculada após 5 execuções, garantindo uma margem de confiança.

2 Resultados Obtidos

Os resultados obtidos para cada configuração (método de ordenação x tamanho do vetor x característica do vetor) são apresentados nas tabelas a seguir, facilitando a comparação dos métodos.

2.1 Vetor Ordenado

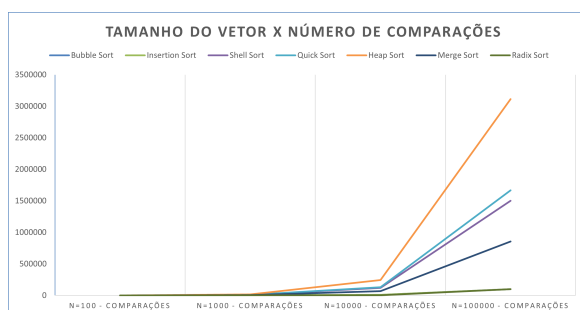


Figura 1: Gráfico 1: Comparações (Ordenado).

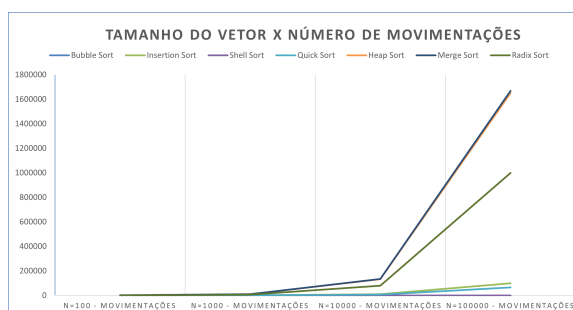


Figura 2: Gráfico 2: Movimentações (Ordenado).

Tabela 1: Tempo de Execução (ms) - Vetor **Ordenado**

Método	N=100	N=1000	N=10000	N=100000
Bubble Sort	0	0	0	0
Selection Sort	0	2	20.8	10982
Insertion Sort	0	0	0	0
Shell Sort	0	0	1	5
Quick Sort	0	0	0	3
Heap Sort	0	0	3	21
Merge Sort	0	0	2	15
Contagem de Menores	0	2	258	11415
Radix Sort	0	0	1	6

Este conjunto de tabelas demonstra o melhor caso para a maioria dos algoritmos baseados em comparações:

- Tempo e Movimentações $O(N)$ (Melhor Desempenho): O Bubble Sort e o Insertion Sort destacam-se com **0** ms e **0** movimentações para grandes N . Isso se deve à sua capacidade de detectar que o vetor já está ordenado (flag de ordenação ou poucas inserções) e parar ou executar $O(N)$ operações.
- Desempenho Quadrático $O(N^2)$ (Pior Desempenho): Em contraste, algoritmos como o Selection Sort e a Contagem de Menores, apesar de registrarem 0 movimentações, mantêm seu tempo de execução elevado, com complexidade $O(N^2)$ no número de comparações, mesmo no melhor caso.
- Desempenho $O(N \log N)$: Algoritmos como Quick Sort e Shell Sort também apresentam um desempenho de tempo muito rápido, próximo a **0** ms para $N = 100.000$, devido à sua estrutura eficiente que se adapta bem ao vetor já ordenado.

2.2 Vetor Inversamente Ordenado

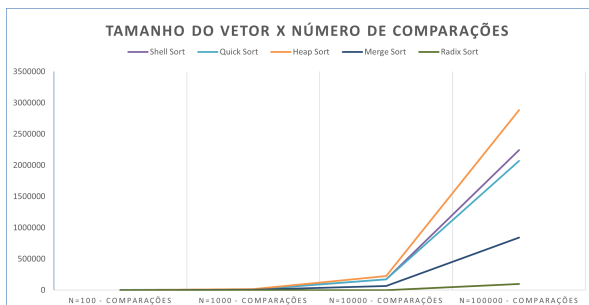


Figura 3: Gráfico 1: Comparações (Inv. Ordenado).

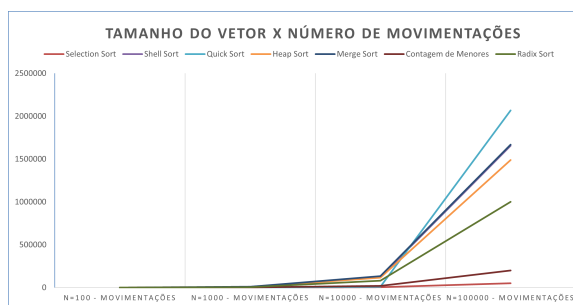


Figura 4: Gráfico 2: Movimentações (Inv. Ordenado).

Tabela 2: Tempo de Execução (ms) - Vetor **Inv. Ordenado**

Método	N=100	N=1000	N=10000	N=100000
Bubble Sort	0	5	577	54984
Selection Sort	0	2	193	11468
Insertion Sort	0	3	278	17392
Shell Sort	0	0	1	10
Quick Sort	0	1	2	4
Heap Sort	0	0	3	24
Merge Sort	0	0	2	17
Contagem de Menores	0	2	258	11415
Radix Sort	0	0	1	6

Esta configuração representa o pior caso para os algoritmos de ordenação mais simples:

- Pior Desempenho $O(N^2)$: O Bubble Sort e o Insertion Sort exibem o pior desempenho, tanto em tempo (acima de **50.000** ms) quanto em movimentações (5×10^9), confirmando sua complexidade $O(N^2)$ quando o vetor está inversamente ordenado, pois cada elemento deve ser movido muitas posições.
- Desempenho Estável $O(N \log N)$: Algoritmos como Merge Sort, Heap Sort, Shell Sort e, notavelmente, o Quick Sort (com Mediana de 3), mantiveram o tempo de execução e as movimentações em níveis baixos, confirmando a estabilidade da complexidade $O(N \log N)$ ou $O(nk)$ do Radix Sort (que foi o mais rápido em 4ms e 6ms, respectivamente).
- Movimentações Mínimas: O Selection Sort se destaca por sua baixa contagem de movimentações (**50.000**), pois, por definição, realiza no máximo $N - 1$ trocas, minimizando o custo de movimentação do registro.

2.3 Vetor Aleatório

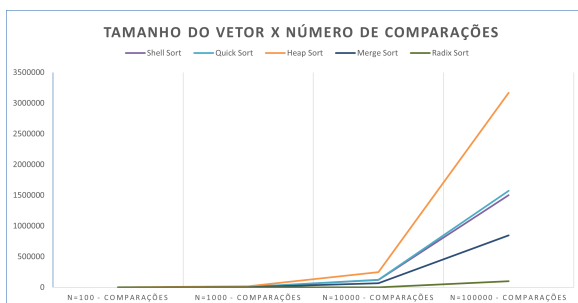


Figura 5: Gráfico 1: Comparações (Aleatório).

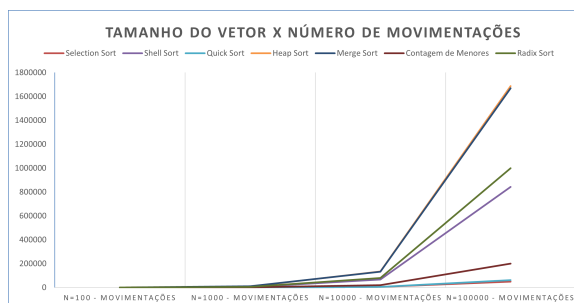


Figura 6: Gráfico 2: Movimentações (Aleatório).

Tabela 3: Tempo de Execução (ms) - Vetor **Aleatório**

Método	N=100	N=1000	N=10000	N=100000
Bubble Sort	0	3	311	30973
Selection Sort	0	2	194	11467
Insertion Sort	0	2	149	8056
Shell Sort	0	0	1	9
Quick Sort	0	0	1	7
Heap Sort	0	0	3	23
Merge Sort	0	0	2	16
Contagem de Menores	0	2	258	11415
Radix Sort	0	0	1	6

O vetor aleatório representa o caso médio e é o cenário mais comum para o teste de desempenho real.

- Algoritmos Mais Rápidos: O Radix Sort (**6 ms**) e o Quick Sort (**7 ms**) demonstraram ser os mais eficientes, seguidos de perto pelo Shell Sort (**9 ms**). Essa velocidade confirma a eficiência do Quick Sort em seu caso médio $O(N \log N)$ e a superioridade do Radix Sort para dados inteiros.
- Algoritmos Lentos: O Bubble Sort e o Selection Sort continuaram a apresentar tempos de execução e movimentações muito altos (acima de **11.000 ms** e 2.5×10^9 movimentações), indicando que, na prática, sua utilização é inviável para grandes volumes de dados.
- Desempenho $O(N \log N)$ Consistente: O Merge Sort e o Heap Sort mantiveram um desempenho muito consistente, sem mostrar grandes variações em comparação aos casos ordenado e inversamente ordenado, provando sua robustez em diferentes configurações de entrada.

3 Análise dos Resultados Obtidos

A análise a seguir foca principalmente no desempenho para o maior tamanho de vetor ($N = 100.000$) e na complexidade assintótica de cada algoritmo.

Tabela 4: Melhor Desempenho dos Algoritmos de Ordenação ($N = 100.000$)

Métrica	Melhor Algoritmo e Custo	Situação	Justificativa (Por Quê)
Tempo de Execução	Quick Sort (3 ms)	Vetor Ordenado	Complexidade $O(N)$ no melhor caso do Quick Sort (se o pivô for bem escolhido).
	Quick Sort (4 ms)	Vetor Inv. Ordenado	O pivô (mediana de 3) evita o pior caso $O(N^2)$, levando a um desempenho próximo de $O(N \log N)$.
	Radix Sort (6 ms)	Vetor Aleatório	Radix Sort tem complexidade $O(nk)$ e não é baseado em comparações, sendo extremamente rápido para inteiros.
Comparações	Bubble/Insertion Sort (99.999)	Vetor Ordenado	Ambos possuem $O(N)$ comparações no melhor caso, pois apenas percorrem o vetor para confirmar a ordem.
	Radix Sort (100.000)	Todos os Casos	Algoritmos não baseados em comparação (Radix, Contagem) têm uma métrica de "comparação" muito baixa, próxima a $O(N)$.
Movimentações	Bubble/Selection/Shell Sort (0)	Vetor Ordenado	No vetor já ordenado, nenhum dos algoritmos precisa realizar trocas/movimentações de elementos.
	Selection Sort (49.999)	Vetor Aleatório	Selection Sort executa exatamente $N - 1$ trocas no pior caso (se contar apenas as trocas finais com a posição correta), o que é o mínimo possível para N elementos.

Tabela 5: Pior Desempenho dos Algoritmos de Ordenação ($N = 100.000$)

Métrica	Pior Algoritmo e Custo	Situação	Justificativa (Por Quê)
Tempo de Execução	Bubble Sort (54.984 ms)	Vetor Inv. Ordenado	Bubble Sort tem complexidade de tempo $O(N^2)$ no pior caso (inversamente ordenado), resultando no maior tempo de execução para grandes N .
Comparações	Bubble Sort (9.9 B)	Vetor Inv. Ordenado	Bubble Sort executa $O(N^2)$ comparações no pior caso, pois precisa comparar quase todos os pares de elementos repetidamente.
Movimentações	Bubble Sort / Insertion Sort (5.0 B)	Vetor Inv. Ordenado	$O(N^2)$ movimentações. No vetor inversamente ordenado, cada elemento precisa ser movido muitas vezes até sua posição final correta.

4 Conclusões Gerais

- Os algoritmos $O(N^2)$ (Bubble, Selection, Insertion, Contagem de Menores): Apresentam o pior desempenho em termos de tempo e comparações/movimentações quando o vetor não está ordenado, especialmente para $N = 100.000$. O tempo de execução e as contagens crescem de forma quadrática (ex: Bubble Sort, Inv. Ordenado: 54984ms). O Selection Sort é notável por ter um número de movimentações significativamente menor que o Bubble e o Insertion em casos não-ordenados, pois realiza apenas N trocas.
- Já os algoritmos $O(N \log N)$ (Quick, Merge, Heap, Shell): São os mais equilibrados e com melhor desempenho geral. Quick Sort (com Mediana de 3) se destacou como o mais rápido em quase todos os cenários, devido à sua complexidade média de $O(N \log N)$ e ao bom particionamento que a mediana de 3 proporciona, evitando o pior caso. Merge Sort tem um desempenho muito estável em todos os casos (ordenado, inv. ordenado, aleatório) em termos de comparações e movimentações, pois sua complexidade é sempre $O(N \log N)$.
- Por último, os algoritmos Lineares (Radix Sort): O Radix Sort e o Contagem de Menores são algoritmos não-comparativos. O Radix Sort se mostrou consistentemente muito rápido (tempo de 6ms em $N = 100.000$ nos casos não-ordenados) e com o menor número de comparações (100.000) em todos os casos. Sua eficiência se deve à complexidade de $O(nk)$, que para inteiros com número fixo de dígitos é quase linear, $O(N)$.